

Applied Machine Learning
COMP 551 001
December 14 9:00 am – 12:00 pm

EXAMINER: Yue Li ASSOC. EXAMINER: Reihaneh Rabbany

STUDENT NAME:		McGILL ID:							
---------------	--	------------	--	--	--	--	--	--	--

EXAM:	CLOSED BOOK ☐		OPEN BOOK ☒	
	PRINTED ON BOTH SIDES OF THE PAGE ☐		SINGLE-SIDED ☒	
	MULTIPLE CHOICE ANSWER SHEETS: YES ☐ NO ☒			
	NOTE: The Examination Security Monitor Program detects pairs of students with unusually similar answer patterns on multiple-choice exams. Data generated by this program can be used as admissible evidence, either to initiate or corroborate an investigation or a charge of cheating under Section 17 of the Code of Student Conduct and Disciplinary Procedures.			
	ANSWER BOOKLET REQUIRED: YES ☐ NO ☒			
	EXTRA BOOKLETS PERMITTED: YES ☐ NO ☒			
	ANSWER ON EXAM: YES ☒ NO ☐			
	SHOULD THE EXAM BE: RETURNED ☒ KEPT BY STUDENT ☐			
CRIB SHEETS:	PERMITTED ☒ NOT PERMITTED ☐			
DICTIONARIES:	TRANSLATION ☒ REGULAR ☒ NOT PERMITTED ☐			
CALCULATORS:	NOT PERMITTED ☐ PERMITTED (Non-Programmable) ☒ PERMITTED (Programmable) ☐			
ANY SPECIAL INSTRUCTIONS:				

Course: COMP 551 Fall 2022

Page 1 of 13

- You have 180 minutes to write the exam.
- Hard-copy notes, books, and printed slides are allowed but electronic devices are NOT allowed.
- This exam contains 24 questions on 18 pages for total of 100 + 2 bonus points.
- 10 multiple-choice questions: circle only ONE correct answer per question.
- 4 multiple-select questions, circle ALL correct answers per question. Scoring: Right minus Wrong.
- 9 short-answer questions (including a bonus): write your answer directly below each question.
- Advice: Try not to spend too much time searching answers through your notes as it will slow you down and you will not have enough time to complete this exam in 3 hours. Good luck!

1 Multiple-choice questions (30 points)

- (3 points) Of the following supervised learning methods, which one *always* requires the presence of the *entire* training data in order to make prediction on any test data?
 - A. K nearest neighbour
 - B. Decision tree
 - C. Logistic regression
 - D. MLP
 - E. Support vector machine
- (3 points) In the context of bias-variance trade-off, which of the following actions will *most likely* decrease *both* the model variance and the bias?
 - A. Increase tree-depth in the regression tree.
 - B. Increase regularization in ridge regression.
 - C. Increase K in K -nearest neighbours.
 - D. Increase the number of hidden units in each layer of the MLP.
 - E. Increase the number of trees in Random Forest.
- (3 points) Suppose we have a function $\hat{y} = f(\mathbf{x}; \theta)$, where $\mathbf{x} \in \mathcal{R}^D$, $\hat{y} \in \mathcal{R}$ (i.e., $\hat{y}$ is a real number), and θ are the learnable model parameters. To train this model for a *binary classification* task, choose the correct *loss function* below.
 - A. $\mathcal{L} = (\hat{y} - y)^2$
 - B. $\mathcal{L} = -y \log \hat{y} - (1 - y) \log (1 - \hat{y})$
 - C. $\mathcal{L} = y \log \sigma(\hat{y}) + (1 - y) \log (1 - \sigma(\hat{y}))$, where $\sigma(\hat{y}) = (1 + \exp(-\hat{y}))^{-1}$
 - D. $\mathcal{L} = -y \log \sigma(\hat{y}) - (1 - y) \log (1 - \sigma(\hat{y}))$, where $\sigma(\hat{y}) = (1 + \exp(-\hat{y}))^{-1}$
- (3 points) After training a binary classifier that can produce probability for the positive class, what threshold (if any) guarantees to produce 100% *True Positive Rate* on the test data? Note we set the predicted class to 1 if the model predicted probability is *greater* than the threshold.
 - A. -1
 - B. 0.5
 - C. 1
 - D. No such threshold exists

COMP 551 Fall 2022 Final Exam

Page 2 of 18

Solution: For $\text{TPR} = \text{TP} / (\text{TP} + \text{FN})$, we can have TPR equal to 1 when the threshold is -1. That is, every data point is predicted to be positive and have zero false negative.

- (3 points) Suppose we have a training dataset with 1000 features but only 50 training examples. What's the main concern in choosing a deep neural network to fit such data?
 - A. Overfitting
 - B. Underfitting
 - C. Model convergence
 - D. Hyperparameter tuning
 - E. Vanishing gradient
 - F. Gradient explosion
- (3 points) What technique is the key that allows us to efficiently train a neural network on millions of training examples?
 - A. Batch normalization
 - B. Bayesian optimization
 - C. Regularization
 - D. ReLU activation
 - E. Stochastic gradient descent
- (3 points) How many parameters in total for a 2-layer MLP of 9 input units, 5 hidden units in each layer and 3 output units? Assuming there is a bias term for each hidden unit and output unit.
 - A. 85
 - B. 94
 - C. 98
 - D. 135

Solution: Correct: $(9 + 1) \times 5 + (5 + 1) \times 5 + (5 + 1) \times 3 = 50 + 30 + 18 = 98$
Incorrect: $9 \times 5 + 5 \times 5 + 5 \times 3 = 45 + 25 + 15 = 85$

- (3 points) Which model has the *smallest number* of parameters in training to predict whether a 28×28 MNIST image is a written digit "2" (i.e., a binary classifier; assuming no bias)?
 - A. A ConvNet with a single convolutional filter of size 28×28 (no padding or stride) directly followed by a sigmoid transformation to produce output probability $\hat{y}$.
 - B. A ConvNet with a single convolutional filter of size 1×1 (no padding or stride) followed by flattening and then an output layer to produce output probability $\hat{y}$.
 - C. A 1-layer MLP with 1 hidden unit in its hidden layer followed by a sigmoid transformation to produce output probability $\hat{y}$.

Solution:

- The ConvNet with a 28×28 filter has parameters size equal to 28×28

- The ConvNet with a 1×1 filter need to have a dense layer of $(28 \times 28) \times 1$ and therefore has parameters $1 + 28 \times 28$
- The 1-layer MLP has $28 \times 28 = 784$ input-to-hidden and then a 1 parameter to go from hidden to output and therefore also has $28 \times 28 + 1$ parameters

- (3 points) Suppose we are designing a recurrent neural network (RNN) for diagnosing patients after 5 days during their 10-day hospital stay. That is, the RNN model takes a sequence of input $\mathbf{x}_1, \dots, \mathbf{x}_5$ and makes prediction for $\mathbf{y}$ at the end of the fifth day. Which of the following will *not* increase the number of parameters in such RNN?
 - A. Increasing the number of hidden units
 - B. Instead of 5 days, we decide to let the RNN train on 7-day of the time-series data to more accurately diagnose patients
 - C. Increasing the number of hidden layers
 - D. Adding more static features such as patient age, sex, and pre-existing medical conditions.
 - E. All of the above will increase the number of parameters for the RNN

Solution: Because the weights are shared from time t to $t + 1$, increasing the length of the input sequence will not increase the RNN parameters

- (3 points) Suppose you are training a LASSO model with the penalty tuning parameter λ set to 1 on a training data with $N = 10$ training examples. Each input feature is *standardized* to have mean equal to 0 and standard deviation equal to 1. After iteration $t - 1$, the value for the estimated coefficient $\hat{w}_d^{(t-1)}$ for feature d is 0.5. *Without the ℓ_1 penalty*, the partial derivative of the Sum of Squared Error (SSE) w.r.t. w_d is 0.1 (while fixing all other coefficients $\hat{\mathbf{w}}_{\setminus d}^{(t-1)}$). That is, $-\mathbf{x}_d^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d \|\mathbf{x}_d\|_2^2 = 0.1$ where $\hat{\mathbf{y}}_{\setminus d} = \mathbf{X} \hat{\mathbf{w}}_{\setminus d}^{(t-1)}$. Recall in Module 4.5 that the LASSO update rule is:

$$\hat{w}_d = \begin{cases} \frac{\mathbf{x}_d^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + \lambda}{\|\mathbf{x}_d\|_2^2} & \mathbf{x}_d^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) < -\lambda \\ 0 & \mathbf{x}_d^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) \in [-\lambda, \lambda] \\ \frac{\mathbf{x}_d^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) - \lambda}{\|\mathbf{x}_d\|_2^2} & \mathbf{x}_d^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) > \lambda \end{cases}$$

What is the new LASSO estimate for $\hat{w}_d^{(t)}$ at iteration t ?

- A. -0.49
- B. -0.39
- C. 0
- D. 0.39
- E. 0.49

Solution: To apply the update LASSO rule, we need to compute $\mathbf{x}_d^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d})$. Since the data are standardized, we know that $\|\mathbf{x}_d\|_2^2 = N = 10$. Therefore, $\mathbf{x}_d^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) = 4.9 > \lambda = 1$. Applying the third scenario in the update rule, $\hat{w}_d^{(t)} = (4.9 - 1) / 10 = 0.39$.

2 Multiple-select questions (20 points)

11. (4 points) What regression model(s) below is or are equivalent to a model, which simply takes the average of the response values in the training data as the predicted value for all test data points?
- Regression tree with only the root node**
 - K-nearest neighbours with K set to be 1
 - K-nearest neighbours with K set to be the number of training examples**
 - Linear regression that only fits the bias term, i.e., $\hat{y}^{(n)} = w_0$.**
12. (4 points) Circle ALL method(s) that can be properly trained by gradient descent.
- Decision tree
 - Linear regression without regularization**
 - Linear regression with ℓ_1 -regularization
 - Logistic regression with ℓ_2 -regularization**
13. (4 points) Choose ALL technique(s) that can reduce the number of parameters of a convolutional neural network (considering everything else fixed).
- Decrease the number of convolutional filters**
 - Increase stride size at every convolutional layer**
 - Increase average pooling size at each convolutional layer**
 - Decouple the filter of dimension $H \times W \times C \times D$ for height, width, channel, and depth (output channel) by using a 2D filter W of dimension $H \times W$ and another 2D filter of dimension $C \times D$ to perform depthwise separable convolution.**
14. (4 points) We are training a recurrent neural network (RNN) on IMDB movie review data to predict good and bad movies by treating each review as sequential text data. Choose ALL plausible cause(s) of vanishing gradient.
- Too many hidden layers in the RNN.**
 - Too many hidden units at each hidden layer in the RNN.
 - Some movie reviews are too long.**
 - Sigmoid function is used to activate each hidden unit.**
15. (4 points) Connect each problem on the left to the most suitable deep learning method on the right.
- | | |
|--|---|
| • Predicting spams using hand-crafted keyword features | • RNN for Seq2Seq unaligned case |
| • Translating English sentences to French sentences | • MLP with sigmoid output |
| • Predicting cancer stages from tumor images | • RNN for Seq2Vec |
| • Name entity recognition in sentences | • RNN for Seq2Seq aligned case |
| • Predicting content categories from unstructured text reports | • Convolutional neural network + MLP with softmax outputs |

Solution:

- Predict spams using handcrafted keyword features, MLP with sigmoid output
- Translating English sentence to French sentence, RNN for Seq2Seq unaligned case
- Predicting cancer stages from tumor images, Convolutional neural network + MLP with softmax outputs
- Name entity recognition in sentences, RNN for Seq2Seq aligned case
- Predicting content categories from the unstructured text reports, RNN for Seq2Vec

3 Short answer questions (50 points)

16. (2 points) When performing Stochastic Gradient Descent (SGD) using a simplified version of Adams update as shown below,
- (1 points) What happens if we increase β_1 in the update equation below?
 - (1 points) What happens if we increase β_2 in the update equation below?

$$\begin{aligned}\mathbf{M}^{(t)} &\leftarrow \beta_1 \mathbf{M}^{(t-1)} + (1 - \beta_1) \nabla J(\mathbf{w}^{(t-1)}) \\ \mathbf{S}^{(t)} &\leftarrow \beta_2 \mathbf{S}^{(t-1)} + (1 - \beta_2) \left(\nabla J(\mathbf{w}^{(t-1)}) \right)^2 \\ \mathbf{w}^{(t)} &\leftarrow \mathbf{w}_d^{(t-1)} - \frac{\alpha}{\sqrt{\mathbf{S}^{(t)} + \epsilon}} \mathbf{M}^{(t)}\end{aligned}$$

Solution: Increasing β_1 means increasing momentum [0.5 pts] and smoother path with less oscillation in SGD [0.5 pts]. Increasing β_2 means preserving more of the earlier changes in the learning step [1 pts].

17. (2 points) In Group LASSO, we have the L2-norm to penalize each group of coefficients:

$$J(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \sum_{g=1}^G \|\mathbf{w}_g\|_2, \quad \text{where} \quad \|\mathbf{w}_g\|_2 = \sqrt{\sum_{d \in S_g} w_d^2}$$

What difference does it make if we used *squared* L2-norm $\sum_{g=1}^G \|\mathbf{w}_g\|_2^2$ instead of L2-norm as the weight penalty in the Group LASSO?

Solution: The Group LASSO reduces to ridge regression when squared L2-norm is used as the weight penalty because $\sum_{g=1}^G \|\mathbf{w}_g\|_2^2 = \sum_{g=1}^G \sum_{d \in S_g} w_d^2 = \sum_{d=1}^D w_d^2$

18. (2 points) When training a multi-class prediction model, some of the class scores computed by $a_c = \mathbf{x}_c \cdot \mathbf{w}_{c,c}$ for $c \in \{1, \dots, C\}$ can be a large positive number (e.g., $a_c = 1000$), which can result in infinity value on the computer when applying the softmax transformation to get predicted class probabilities (i.e., $\exp(a_c) / \sum_{c'} \exp(a_{c'})$). How to resolve this issue and still obtain the same class probabilities as if our computers could handle the exponential of any large number?

Solution: Before applying softmax, we can subtract each class score by the sum or max of the class scores. This is the same as dividing both the numerator and the denominator by a constant such that the class probabilities will not change:

$$\frac{\exp(a_c - \sum_k a_k)}{\sum_{c'} \exp(a_{c'} - \sum_k a_k)} = \frac{\exp(a_c) / \exp(\sum_k a_k)}{\sum_{c'} \exp(a_{c'}) / \exp(\sum_k a_k)} = \frac{\exp(a_c)}{\sum_{c'} \exp(a_{c'})}$$

[YL: If they said they divide top and bottom with a large number like 1000, give them one point.]

19. (8 points) Describe an MLP that is equivalent to a multiple linear regression model $\hat{y} = \sum_{d=1}^{100} \phi_d(x) w_d$ that operates on 100 non-linear basis features computed from 100 Gaussian basis functions, each of which transforms the same 1D feature x by:

$$\phi_d(x) = \exp\left(-\frac{x^2}{2\sigma_d^2}\right), \quad \text{where} \quad \sigma_d = \sqrt{d/2}$$

Ignore the bias in this question.

- (2 pts) What's the architecture of such MLP?
- (2 pts) What are the input-to-hidden weights in the MLP?
- (2 pts) What's the activation function for each hidden unit?
- (2 pts) What are the hidden-to-output weights?

Solution:

- (2 pts) This is equivalent to a 1-layer MLP with 100 hidden units and an output unit.
- (2 pts) The weight from the 1D feature x to each hidden unit is $v_d = \frac{1}{\sqrt{d}}$ because

$$\frac{x^2}{2\sigma_d^2} = \frac{x^2}{2\frac{d}{2}} = \frac{x^2}{d} = \left(\frac{x}{\sqrt{d}}\right)^2 \triangleq (v_d x)^2 \triangleq a_d^2$$

- (2 pts) Given $a_d = v_d x$, the activation function for each hidden unit is:

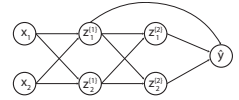
$$f(v_d x) = \exp(-a_d^2)$$

- (2 pts) The hidden-to-output weights are the same as the 200 linear regression coefficients $\{w_d\}_{d=1}^D$.

We can accept any other equivalent answer.

20. (10 points) The computational graph below illustrates the forward pass of a small neural network:

$$\begin{aligned} z_i^{(1)} &= \sigma(x_1 w_{1,i}^{(1)} + x_2 w_{2,i}^{(1)}) \quad \forall i \in \{1, 2\} \\ z_j^{(2)} &= \sigma(z_1^{(1)} w_{1,j}^{(2)} + z_2^{(1)} w_{2,j}^{(2)}) \quad \forall j \in \{1, 2\} \\ \hat{y} &= z_1^{(2)} w_1^{(o)} + z_2^{(2)} w_2^{(o)} + z_1^{(1)} w_3^{(o)} \\ \mathcal{L} &= (y - \hat{y})^2 \end{aligned}$$

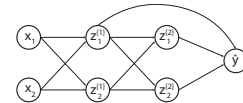


where $\sigma(x)$ is the sigmoid function. Assume we are performing automatic differentiation.

- (5 pts) Clearly write down the steps needed to obtain the partial derivative of $\frac{\partial \mathcal{L}}{\partial z_1^{(1)}}$ in the *forward mode*

- (5 pts) Clearly write down the steps needed to obtain $\frac{\partial \mathcal{L}}{\partial z_1^{(1)}}$ in the *backward mode*

Solution: [YL: For this question, it's ok if they write the expressions and recycle them in the later gradient chain rule as long as they expressed all of them in analytical forms.]



$$\begin{aligned} z_i^{(1)} &= \sigma(x_1 w_{1,i}^{(1)} + x_2 w_{2,i}^{(1)}) \quad \forall i \in \{1, 2\} \\ z_j^{(2)} &= \sigma(z_1^{(1)} w_{1,j}^{(2)} + z_2^{(1)} w_{2,j}^{(2)}) \quad \forall j \in \{1, 2\} \\ \hat{y} &= z_1^{(2)} w_1^{(o)} + z_2^{(2)} w_2^{(o)} + z_1^{(1)} w_3^{(o)} \\ \mathcal{L} &= (y - \hat{y})^2 \end{aligned}$$

- Forward mode (from left to right):

$$\begin{aligned} \frac{\partial z_1^{(1)}}{\partial z_1^{(1)}} &= 1 \\ \frac{\partial z_1^{(2)}}{\partial z_1^{(1)}} &= z_1^{(2)} (1 - z_1^{(2)}) w_{1,1}^{(2)} \\ \frac{\partial z_2^{(2)}}{\partial z_1^{(1)}} &= z_2^{(2)} (1 - z_2^{(2)}) w_{1,2}^{(2)} \\ \frac{\hat{y}}{\partial z_1^{(1)}} &= \frac{\partial z_1^{(2)}}{\partial z_1^{(1)}} w_1^{(o)} + \frac{\partial z_2^{(2)}}{\partial z_1^{(1)}} w_2^{(o)} + \frac{\partial z_1^{(1)}}{\partial z_1^{(1)}} w_3^{(o)} \\ \frac{\partial \mathcal{L}}{\partial z_1^{(1)}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1^{(1)}} = -2(y - \hat{y}) \frac{\hat{y}}{\partial z_1^{(1)}} \\ &= -2(y - \hat{y}) \left(z_1^{(2)} (1 - z_1^{(2)}) w_{1,1}^{(2)} w_1^{(o)} + z_2^{(2)} (1 - z_2^{(2)}) w_{1,2}^{(2)} w_2^{(o)} + w_3^{(o)} \right) \end{aligned}$$

- Backward mode (from right to left):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \hat{y}} &= -2(y - \hat{y}) \\ \frac{\partial \mathcal{L}}{\partial z_1^{(2)}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1^{(2)}} = -2(y - \hat{y}) w_1^{(o)} \\ \frac{\partial \mathcal{L}}{\partial z_2^{(2)}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2^{(2)}} = -2(y - \hat{y}) w_2^{(o)} \\ \frac{\partial \mathcal{L}}{\partial z_1^{(1)}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1^{(1)}} + \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial z_1^{(1)}} + \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial z_1^{(1)}} \\ &= -2(y - \hat{y}) \left(z_1^{(2)} (1 - z_1^{(2)}) w_{1,1}^{(2)} w_1^{(o)} + z_2^{(2)} (1 - z_2^{(2)}) w_{1,2}^{(2)} w_2^{(o)} + w_3^{(o)} \right) \end{aligned}$$

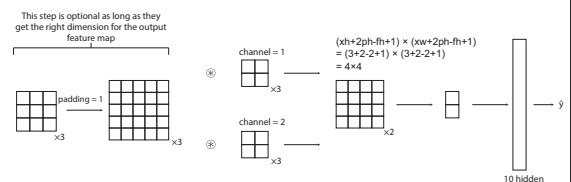
21. (10 points) Suppose we are designing a convolutional neural network to convolve 3×3 color images of 3 color channels. For each input color channel and output channel, our ConvNet has a unique filter with size of 2×2 . The padding of size 1 is applied to each color channel of the input image. No stride or dilation is applied. ReLU activation is applied to the outputs of the linear convolution.

The ConvNet has two output channels. A global max-pooling is applied to the convolved feature map. The results of the global pooling are then fed into a dense layer of 10 hidden units followed by a sigmoid output unit for prediction (e.g., whether the image contains a cat).

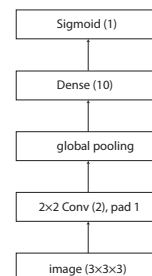
- (5 pts) Based on the above description, draw the ConvNet architecture in detailed view.
- (3 pts) Draw the same ConvNet in the compact view.
- (2 pts) Assuming no bias term, how many learnable parameters are in this ConvNet?

Solution:

- Detailed view:



- Compact view:

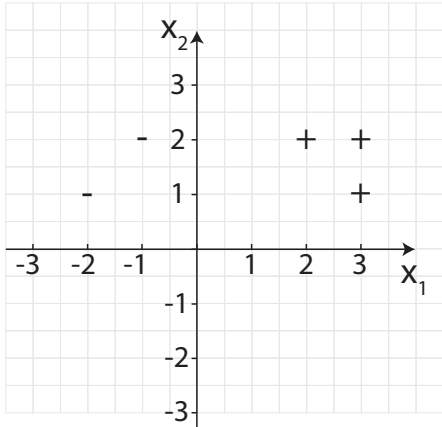


- # Parameters: $3 \times 2 \times 2 \times 2 + 2 \times 10 + 10 = 54$

22. (6 points) A Perceptron trained to separate 2D data points (x_1, x_2) has parameters set to be $w_0 = -2$ (bias), $w_1 = 4$ for x_1 , and $w_2 = 2$ for x_2 .

- (3 pts) Compute the decision boundary of the Perceptron and draw it directly on the plot below.
- (3 pts) Suppose the training examples are $\{(-2, 1), -1\}$, $\{(-1, 2), -1\}$, $\{(2, 2), +1\}$, $\{(3, 1), +1\}$, and $\{(3, 2), +1\}$ (in the format of $\{(x_1^{(n)}, x_2^{(n)}), \tilde{y}^{(n)}\}$) as shown on the plot below. What are the
 - support vectors (1 pt),
 - margins (1 pt),
 - and decision boundary (1 pt)

determined by the hard-margin SVM classifier? Write down and draw your answers on the plot.



Solution:

- Its decision boundary is defined by all pairs of x_1 and x_2 . We can solve the Perceptron equation $w_0 + w_1x_1 + w_2x_2 = 0$ for either x_1 or x_2 :

$$x_1 = -\frac{w_0}{w_1} - \frac{w_2}{w_1}x_2 = 0.5 - 0.5x_2$$

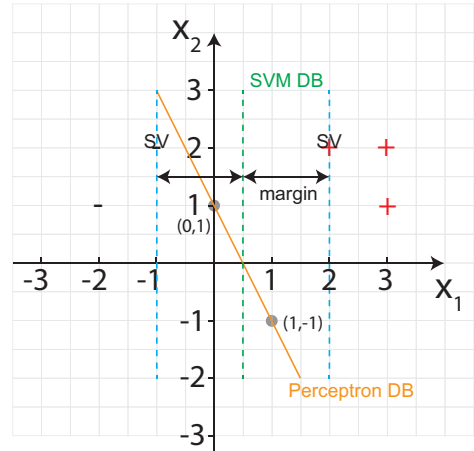
or

$$x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2}x_1 = 1 - 2x_1$$

Given x_1 (or x_2), there is a unique point value for x_2 (or x_1), which defines the decision boundary of Perceptron.

- The SVM hard-margin chooses the training examples at $(-1, 2)$ and $(2, 2)$ as the support vectors to define its margins because they are the closest to the decision boundary that result in the maximum margin of 1.5.

If they have their decision boundary and margin rotated to some degree, it is still correct as long as the margins pass through the same data points and it does not reduce the margin size.



23. (5 points) Describe the transformer encoder block. Highlight the key elements and computation needed to obtain the multi-headed attention (3 pts). Also, describe how we can go deeper by stacking transformer encoder blocks (2 pts). You may use equations to provide concise answer.

Solution: Let i denote the index of the attention head and H_ℓ be the number of attention heads at layer ℓ . When $\ell = 1$, $\mathbf{Z}^{(1)} = \mathbf{X}$. Word embedding from the hidden layer $\ell - 1$ to hidden layer ℓ are computed as follows:

Query: $\mathbf{Q}_i^{(\ell)} = \mathbf{Z}^{(\ell-1)} \mathbf{W}_i^{(q)}$; Key: $\mathbf{K}_i^{(\ell)} = \mathbf{Z}^{(\ell-1)} \mathbf{W}_i^{(k)}$; Value: $\mathbf{V}_i^{(\ell)} = \mathbf{Z}^{(\ell-1)} \mathbf{W}_i^{(v)}$

Self-attention: $\mathbf{A}_i^{(\ell)} = \text{Attn}(\mathbf{Q}_i^{(\ell)}, \mathbf{K}_i^{(\ell)})$; Word embedding from head i : $\mathbf{Z}_i^{(\ell)} = \mathbf{A}_i^{(1)} \mathbf{V}_i^{(1)}$

Concatenating the embedding from all heads:

$$\mathbf{Z}^{(\ell)} = [\mathbf{Z}_1^{(\ell)}, \dots, \mathbf{Z}_{H_\ell}^{(\ell)}]$$

24. (5 points) Briefly describe with one sentence each the pre-training tasks of Word2vec skip-gram (1 pt), Embeddings from Language Models (ELMo) (1 pt), Generative Pre-trained Transformer (GPT) (1 pt), Bidirectional Encoder Representations from Transformers (BERT) (1 pt), and an example of a fine-tuning task after pre-training by any of the above models (1 pt).

Solution:

- Word2vec skip-gram predicts the flanking words based on the middle words via an MLP.
- ELMo pre-trains by predicting each word based on both the past and future sequence using a bi-directional LSTM.
- GPT pre-trains by predicting the next word a transformer decoder in an auto-regressive way using the past sequence.

- BERT predicts randomly masked words and/or next sentence using transformer-based self-attention.

- Sentiment prediction, NER, sentence pair entailment, neural machine translation, and any other reasonable example can be the answer here.

You may use this page as sketch.

You may use this page as sketch.



APRIL 2024
FINAL EXAMINATION

Applied Machine Learning

COMP 551 001

April 22 2:00 pm – 5:00 pm

EXAMINER: Yue Li

ASSOC. EXAMINER: Isabeau Prémont-Schwarz

STUDENT NAME:		McGILL ID:								
---------------	--	------------	--	--	--	--	--	--	--	--

EXAM:	CLOSED BOOK ☐	OPEN BOOK ☒
	SINGLE-SIDED ☐	PRINTED ON BOTH SIDES OF THE PAGE ☐
	MULTIPLE CHOICE ANSWER SHEETS: YES ☒ NO ☐ NOTE: The Examination Security Monitor Program detects pairs of students with unusually similar answer patterns on multiple-choice exams. Data generated by this program can be used as admissible evidence, either to initiate or corroborate an investigation or a charge of cheating under Section 17 of the Code of Student Conduct and Disciplinary Procedures.	
	ANSWER BOOKLET REQUIRED:	YES ☐ NO ☒
	EXTRA BOOKLETS PERMITTED:	YES ☐ NO ☒
	ANSWER ON EXAM:	YES ☒ NO ☐
	SHOULD THE EXAM BE: RETURNED ☒ KEPT BY STUDENT ☐	
CRIB SHEETS:	PERMITTED ☒ Specifications: (ex: one 8 1/2x11 handwritten double-sided sheet) NOT PERMITTED ☐	
DICTIONARIES:	TRANSLATION ONLY ☒ REGULAR ☒ NOT PERMITTED ☐	
CALCULATORS:	NOT PERMITTED ☐ PERMITTED (Non-Programmable) ☒ PERMITTED (Programmable) ☐	
ANY SPECIAL INSTRUCTIONS: e.g. molecular models		

- You have 180 minutes to write the exam.
- Hard-copy notes, books, and printed slides are allowed but electronic devices are NOT allowed.
- This exam contains 24 questions on 18 pages for total of 100.
- 18 multiple-choice questions: answer each question on the SCANTRON sheet provided to you.
- 6 short-answer questions: write your answer directly below each question.
- Advice: Try not to spend too much time searching answers through your notes as it will slow you down and you will not have enough time to complete this exam in 3 hours. Good luck!

1 Multiple-choice questions (54 points)

1. (3 points) You trained a machine learning model to predict the protein content of different food with the mean squared error loss function. Your model has a hyperparameter λ and you need to decide which value of λ to use. You have the following table for the different losses as function of the value of lambda:

λ	training loss	validation loss	test loss
10	30	31	29
3	15	14	16
1	9	16	13
0.3	5	17	18
0.1	2	20	21

Which value of λ should you select

- A. 10
B. 3
C. 1
D. 0.3
E. 0.1

Solution: $\lambda = 3$ minimized the validation loss.

2. (3 points) Looking at the table in question 1, for which value of λ does the model have the highest bias?

- A. 10
B. 3
C. 1
D. 0.3
E. 0.1

Solution: $\lambda = 10$ has the highest training loss.

3. (3 points) In the context of bias-variance trade-off, which of the following actions will *most likely* decrease the model variance?
- Increase tree-depth in the regression tree.
 - Increase the number of attention heads in a Transformer model.
 - Increase K in K -nearest neighbours.**
 - Increase the number of channels in the hidden layer of a CNN.
 - Increase the width of an MLP layer.

Solution: Increasing K in K-NN increases bias and reduces variance.

4. (3 points) In the context of bias-variance trade-off, which of the following actions will *most likely* decrease the model bias?
- Use a smaller training set in K -nearest neighbours.
 - Reduce the size of the hidden state in an RNN.
 - Train for fewer epochs.
 - Increase the L1 regularization strength.
 - Increase the size of a CNN Kernel.**

Solution: Increasing Kernel size makes for a more expressive model, thus reducing bias.

5. (3 points) You are working for a big news agency and you want to build an LLM to automatically tag news stories with a variety of labels. Example tags include: politics, sports, canada, economy, lifestyle. Your set of tags is known in advance and each news story can potentially have multiple tags (for example both "canada" and "politics" for a news story about federal elections). What should the last (output) layer of your model be like?
- Single neuron, Sigmoid activation.
 - Multiple neurons, Softmax activation.
 - Multiple neurons, Sigmoid activation.**
 - Multiple neurons, Linear activation.
 - Multiple neurons, ReLU activation.

Solution: This is a multilabel regression problem.

6. (3 points) For a given threshold value, you obtain the following confusion matrix:

	Predicted Positive	Predicted Negative	Total
Positive	16	4	20
Negative	6	14	20
Total	22	18	40

Where would you put the point for this threshold on the ROC curve (to the nearest decimal point)?

- (0.2, 0.4)
- (0.2, 0.7)
- (0.2, 0.8)
- (0.3, 0.8)**
- (0.3, 0.7)

Solution: The true positive rate is $16/20=0.8$ The false positive rate is $6/20 = 0.3$

7. (3 points) We are building a predictive model for an architecture firm which needs to predict the cost (in dollars) of architectural project proposals for which the firm competes. Which of the following losses should you use to train your model? Let $\hat{y}$ be the prediction of your model and y be the true cost of the project.
- $(y - \hat{y})^2$
 - $-y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$
 - $-\sum_c y_c \log(\hat{y}_c)$
 - $-\sum_c (y_c \log(\hat{y}_c) + (1 - y_c) \log(1 - \hat{y}_c))$
 - $\max(0, 1 - y\hat{y})$

Solution: This is a regression problem, for a continuous value.

8. (3 points) We are training your neural network with SGD and you quickly realize that your gradients are too noisy and you need smoother gradients as a function of epochs. Which of the following will NOT make your gradients smoother?
- Adding or increasing momentum.
 - Using the sum of squared errors rather than mean squared errors as a loss function.**
 - Increasing the minibatch size.
 - Lowering the learning rate.
 - Using Gradient Descent instead SGD.

Solution: Using sum of squared errors rather mean squared errors will multiply your loss function by the batch size, effectively multiplying your learning rate by the batch size which means you will be taking larger gradient steps and thus your gradients will vary more from one step to the next.

9. (3 points) We are training an MLP with SGD. Your MLP is deep to the level that you are suffering from a vanishing gradient problem. Which of the following activation functions could you use in the hidden layers to help alleviate the vanishing gradient problem?
- $\tanh(x) = \frac{\exp(2x) - 1}{\exp(2x) + 1}$
 - $\sigma(x) = \frac{1}{1 + \exp(-x)}$
 - Clipped ReLU: $\text{CReLU}(x) = \min\{1, \max\{0, x\}\}$.**
 - $\log(1 + \exp(a))$

- Sigmoid with a steeper derivative: $\sigma'(x) = \frac{1}{1 + \exp(-100x)}$

Solution: Only

$$\log(1 + \exp(a))$$

(Softplus) does not have a gradient which goes to zero at the extremities (i.e., saturation point).

10. (3 points) We have a training dataset with 1000 features but only 50 training examples on which you are trying to fit a linear regression. Your validation error is extremely large. What could you do to lower the validation error.
- Add more non-linear basis functions.
 - Train using Gradient Descent rather than finding the solution analytically.
 - Lower the learning rate in SGD.
 - Make sure that the dataset is linearly separable.
 - Add an L2 regularization term.**

Solution: You are likely suffering from overfitting, and thus need to reduce variance.

11. (3 points) Which of the following regularization can lead to a solution, where either all D weights are exactly zeros or none of the D weight is exactly 0?
- $\lambda \|\mathbf{w}\|_2^2 = \lambda \sum_{d=1}^D w_d^2$
 - $\lambda \|\mathbf{w}\|_2 = \lambda \sqrt{\sum_{d=1}^D w_d^2}$**
 - $\lambda \|\mathbf{w}\|_1 = \lambda \sum_{d=1}^D |w_d|$
 - $\lambda_1 \|\mathbf{w}\|_2^2 + \lambda_2 \|\mathbf{w}\|_1$
 - $\lambda \sum_{g=1}^G \|\mathbf{w}_g\|_2$, where $\|\mathbf{w}_g\|_2 = \sqrt{\sum_{d \in S_g} w_d^2}$ and $S_g \subset \{1, \dots, D\}$

Solution: $\lambda \|\mathbf{w}\|_2$ is a special case of group lasso where all features form one group. The gradient is undefined when $\sum_d w_d^2 = 0$:

$$\frac{\partial \sqrt{\sum_d w_d^2}}{\partial w_d} = \frac{\partial}{\partial w_d} \left(\sum_d w_d^2 \right)^{1/2} = \frac{w_d}{\sqrt{\sum_d w_d^2}} \quad (\text{undefined when } \sum_d w_d^2 = 0)$$

12. (3 points) Suppose we want to apply L_1 -regularization with weight penalty λ to the weights in an MLP for regression by minimizing sum of squared error loss. We apply non-linear activation functions for all hidden units. Which of the layer weights can we directly optimize using the *coordinate descent* algorithm that we learn in training the LASSO regression?
- input-to-hidden weights
 - hidden-to-hidden weights
 - hidden-to-output weights for regression**

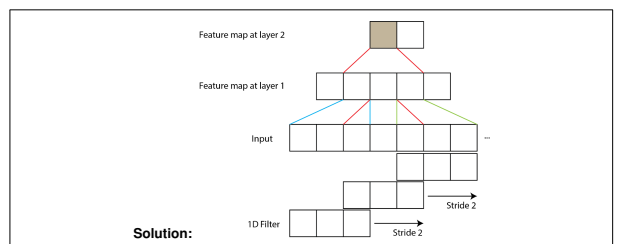
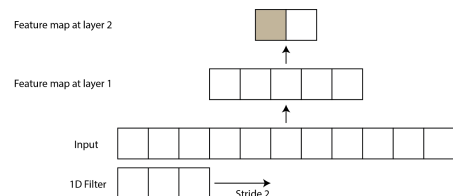
- We cannot apply coordinate descent to any of the layers
- We can apply coordinate descent to *all* weights of the layers**

Solution: Coordinate descent works on hidden-to-output as we can decouple feature from the rest of the features and update the $\hat{\mathbf{w}}_k^{(o)}$ while fixing the rest of the output weights:

$$\hat{\mathbf{w}}_k^{(o)} = \begin{cases} \frac{\mathbf{z}_k^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus k}) + \lambda}{\|\mathbf{z}_k\|_2^2} & \mathbf{z}_k^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus k}) < -\lambda \\ 0 & \mathbf{z}_k^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus k}) \in [-\lambda, \lambda] \\ \frac{\mathbf{z}_k^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus k}) - \lambda}{\|\mathbf{z}_k\|_2^2} & \mathbf{z}_k^T (\mathbf{y} - \hat{\mathbf{y}}_{\setminus k}) > \lambda \end{cases}$$

13. (3 points) We build a 1D CNN, where each convolution layer has kernel size of 3 and a stride of 2. What is the receptive field of each pixel in the 1D output feature map from convolution layer 2 (i.e., the shaded cell in the figure below) with respect to the 1D input?

- 3
- 4
- 5
- 7
- 9



Solution:

14. (3 points) We are using bagging to train an ensemble of models. The training set has 3 samples. Each bagging model will be trained on a different training set. How many possible training sets can models in the ensemble be trained on?
- 1
 - 27
 - 12
 - 5
 - 10

Solution: As shown in the final review session, the number of distinct bootstrap samples can be calculated as $\binom{2N-1}{N-53-10}$. You may also manually enumerate 10 distinct sets as 111, 222, 333, 112, 113, 221, 223, 331, 332, 123.

For this question, there was an error in the original printout. We provided a separate page for this question. We first check the answers on the Scantron. If it is incorrect, then we checked their answer manually on the exam sheet.

15. (3 points) Which of the following statement about boosting is **false**?
- Boosting cannot train weak learners in parallel.
 - Boosting can help reduce model bias.
 - Boosting can work with both regression and classification problems.
 - The m^{th} weak learner is trained to reduce the error made by the ensemble model that uses $m - 1$ weak learners.
- E. Boosting will always work well with any supervised machine learning models by treating them as weak learners.**

Solution: Boosting sequentially trains models to learn the remaining errors, it can thus reduce the bias of the original model. However, boosting relies on combining simple models. It does not work well with highly complex models, which is prone to overfitting. It will not work well with multiple linear models either because of the additive effects of the first multiple linear regression model will make adding another multiple linear regression model completely irrelevant.

16. (3 points) You train an RNN to predict agricultural crop yield (a single number which is determined after the harvest, i.e., seq2vec) by passing the last hidden state of the RNN through a fully connected output layer with linear activation. The RNN has as input the daily weather patterns of the last 60 days before harvest. You get good results but decide to retrain the same model on 90 days of daily weather pattern before harvest instead, in the hope of getting better results. Which of the following statements is **false**?
- The initialization of the hidden state will have less of an effect.
 - The model will use the same amount of memory at test time.
 - The model will backpropagate more gradients.
 - The model will have more trainable parameters.**
 - The vanishing gradient problem becomes more prominent in training RNN on 90-day sequences than in training RNN on 60-day sequences.

Solution: RNN use weight-sharing in time so the model does not have more trainable parameters. The model will use more memory during training because the backpropagation graph will be bigger and take up more memory but will not use more memory at inference time because the hidden state is of constant size.

17. (3 points) What is the loss term that should **not** be in the *full* loss function (including the orthogonal and orthonormal constraints) for deriving the third principal component (PC) $\mathbf{w}_3$ given the first and the second PC? The λ 's below are the Lagrange multipliers.

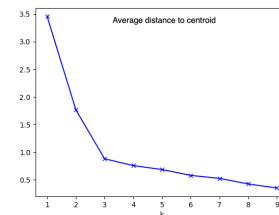
- $\frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - z_{1,n} \hat{\mathbf{w}}_1 - z_{2,n} \hat{\mathbf{w}}_2 - z_{3,n} \mathbf{w}_3\|_2^2$
- $\lambda_3(\mathbf{w}_3^\top \mathbf{w}_3 - 1)$
- $\lambda_{23}(\mathbf{w}_2^\top \mathbf{w}_3)$
- $\lambda_{13}(\mathbf{w}_1^\top \mathbf{w}_3)$
- $\lambda_4 \|\mathbf{w}_3\|_2^2$**

Solution:

$$\mathcal{L}(\mathbf{w}_3) = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - z_{1,n} \hat{\mathbf{w}}_1 - z_{2,n} \hat{\mathbf{w}}_2 - z_{3,n} \mathbf{w}_3\|_2^2 + \lambda_3(\mathbf{w}_3^\top \mathbf{w}_3 - 1) + \lambda_{23}(\mathbf{w}_2^\top \mathbf{w}_3) + \lambda_{13}(\mathbf{w}_1^\top \mathbf{w}_3)$$

Standard PCA does not have the L_2 -regularization constraint.

18. (3 points) You are using K-means clustering. You want to use the elbow method to decide on which is the best k to use so you plot the average distance of points to their centroid as a function of k to get the following graph. Which k should you choose?



- 1
- 3**
- 5
- 7
- 9

Solution: The kink (i.e., inflection point) in the slope happens for k=3.

2 Short answer questions (46 points)

Partial marks were given to the answers that are outside of the rubrics.

19. (5 points) Suppose we design a 1-layer MLP with K hidden units and 1 output for regression. The forward pass for one training data point $\mathbf{x} \in \mathbb{R}^{1 \times D}$ is:

$$\begin{aligned} a_k &= \mathbf{x} \mathbf{w}_k^{(1)} \\ z_k &= \frac{\exp(a_k)}{\sum_{k=1}^K \exp(a_k)} \\ \hat{y} &= \mathbf{z} \mathbf{w}^{(o)} \\ L &= (\hat{y} - y)^2 \end{aligned}$$

Write down the exact mathematical expression of the partial derivative for input-to-hidden weights $w_{d,k}$ for input feature $d \in \{1, \dots, D\}$ and hidden unit $k \in \{1, \dots, K\}$ by the chain rule.

Hint: The activation in the hidden layer is a softmax transformation of the pre-activated hidden activities a_k 's over all K hidden units (whereas the standard MLPs have activation applied to each individual hidden unit, e.g., $z_k = \phi(a_k)$). Therefore, there is a dependency among hidden units due to the softmax transformation – similar to the dependency among the outputs in multi-class regression.

Solution: Accepted answer 1:

$$\frac{\partial L}{\partial w_{d,k}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_k} \sum_{j=1}^K \frac{\partial z_j}{\partial a_k} \frac{\partial a_k}{\partial w_{d,k}}$$

Accepted answer 2 (backprop):

$$\begin{aligned} \frac{\partial L}{\partial \hat{y}} &= 2(\hat{y} - y) \\ \frac{\partial L}{\partial z_k} &= \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_k} \\ \frac{\partial L}{\partial a_k} &= \sum_j \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial a_k} \\ \frac{\partial L}{\partial w_{d,k}} &= \frac{\partial L}{\partial a_k} \frac{\partial a_k}{\partial w_{d,k}} \end{aligned}$$

Here it's OK if they leave the answer like the above without expanding the last partial.

Also, it is OK they wrote down the exact form of the partial for say $\frac{\partial z_j}{\partial a_k}$ but we don't need to deduct point if they got it wrong as long as they got the above partial chain-rule expression right.

Deduct 2 point, if the answer is something like this:

$$\frac{\partial L}{\partial w_{d,k}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_k} \sum_{j=1}^K \frac{\partial z_k}{\partial a_j} \frac{\partial a_j}{\partial w_{d,k}}$$

Deduct 4 points, if the answer is:

$$\frac{\partial L}{\partial w_{d,k}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_k} \frac{\partial z_k}{\partial a_k} \frac{\partial a_k}{\partial w_{d,k}}$$

20. (10 points) Consider the following input $\mathbf{X}$ and convolution filter $\mathbf{W}$.

- (8 pts) [original question] When using zero padding of 1 and stride of 2, what is the output $\mathbf{A}$?
- (8 pts) [updated question] With no zero padding and stride of 1, what is the output $\mathbf{A}$?
- (2 pts) After the convolution layer, a ReLU activation non-linearity is applied followed by a 2×2 max-pooling (i.e., $\text{maxpool}_{2 \times 2}(\text{ReLU}(\mathbf{A}))$). What is the resulting output?

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 1 & 3 \\ 0 & 1 & 0 & 2 \\ 4 & 1 & 1 & 2 \\ 6 & 1 & 3 & 2 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} -1 & 0 & -1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Solution:

[original version]

Assuming the filter stops at the last zero-padded column or row after taking a stride of 2 once.

$$\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 0 & 0 \end{bmatrix} \text{ and } h = [3]$$

[updated version]

$$\mathbf{A} = \begin{bmatrix} 3 & -2 \\ 9 & 0 \end{bmatrix} \text{ and } h = [9].$$

21. (10 points) Suppose we are using a two-headed self-attention transformer module to compute the embeddings of two time points. Input at time 1 is $\mathbf{x}_1 = [0, 1, 0]$ and input at time 2 is $\mathbf{x}_2 = [0, 0, 1]$. Each head $i \in \{1, 2\}$ computes the embedding as:

$$\mathbf{A}_i = \text{softmax}(\mathbf{Q}_i \mathbf{K}_i^\top), \quad \mathbf{Z}_i = \mathbf{A}_i \mathbf{V}_i$$

where $\mathbf{q}_{t,i} = \mathbf{x}_t \mathbf{W}_i^{(query)}$, $\mathbf{k}_{t,i} = \mathbf{x}_t \mathbf{W}_i^{(key)}$, $\mathbf{a}_{t,i} = \text{softmax}(\mathbf{q}_{t,i} \mathbf{K}_i^\top)$ and $\mathbf{v}_{t,i} = \mathbf{x}_t \mathbf{W}_i^{(value)}$; therefore by concatenating rows (i.e., time points), we have $\mathbf{Q}_i = [\mathbf{q}_{1,i}; \mathbf{q}_{2,i}]$, $\mathbf{K}_i = [\mathbf{k}_{1,i}; \mathbf{k}_{2,i}]$, $\mathbf{A}_i = [\mathbf{a}_{1,i}; \mathbf{a}_{2,i}]$, and $\mathbf{V}_i = [\mathbf{v}_{1,i}; \mathbf{v}_{2,i}]$.

For the ease of computation, in this question, we ignore the inverse scaling $\sqrt{D}$ so we are NOT using $\mathbf{A} = f(\mathbf{Q}_i \mathbf{K}_i^\top / \sqrt{D})$ but rather $\mathbf{A} = f(\mathbf{Q}_i \mathbf{K}_i^\top)$ for in this question.

Using the following weights:

$$\mathbf{W}_1^{(query)} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{W}_2^{(query)} = \begin{bmatrix} 0 & -1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{W}_1^{(key)} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ -1 & 0 \end{bmatrix}, \quad \mathbf{W}_2^{(key)} = \begin{bmatrix} 0 & -1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

$$\mathbf{W}_1^{(value)} = \begin{bmatrix} 1 & 0 \\ 1 & -1 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{W}_2^{(value)} = \begin{bmatrix} 0 & -1 \\ 0 & 0 \\ -1 & 1 \end{bmatrix}$$

- (7 pts) Compute the output embedding for either head 1 or head 2 for time step 2 (i.e., to save your time, you don't need to compute embedding for both heads)? There is no need to calculate exponential and you can write your answer in terms of the exponential. Show your work.
- (3 pts) What's the dimension of the output embedding for time 2 if we concatenate the embedding from the two heads? Specify both the number of rows and the number of columns in your answer.

Solution:

1.

$$\mathbf{q}_1 = \mathbf{x}_2 \mathbf{W}_1^{(query)} = [0, 1] \quad (1)$$

$$\mathbf{q}_2 = \mathbf{x}_2 \mathbf{W}_2^{(query)} = [1, 0] \quad (2)$$

$$\mathbf{X} = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{K}_1 = \mathbf{X} \mathbf{W}_1^{(key)} = \begin{bmatrix} 1 & 1 \\ -1 & 0 \end{bmatrix} \quad (3)$$

$$\mathbf{K}_2 = \mathbf{X} \mathbf{W}_2^{(key)} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \quad (4)$$

$$\mathbf{V}_1 = \mathbf{X} \mathbf{W}_1^{(value)} = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \quad (5)$$

$$\mathbf{V}_2 = \mathbf{X} \mathbf{W}_2^{(value)} = \begin{bmatrix} 0 & 0 \\ -1 & 1 \end{bmatrix} \quad (6)$$

$$\mathbf{q}_1 \mathbf{K}_1^\top = [1, 0] \quad (7)$$

$$\mathbf{q}_2 \mathbf{K}_2^\top = [0, 1] \quad (8)$$

$$\mathbf{a}_1 = \text{Softmax}(\mathbf{q}_1 \mathbf{K}_1^\top) = \frac{1}{1+e} [e, 1] = \left[\frac{e}{1+e}, \frac{1}{1+e} \right] \quad (9)$$

$$\mathbf{a}_2 = \text{Softmax}(\mathbf{q}_2 \mathbf{K}_2^\top) = \frac{1}{1+e} [1, e] = \left[\frac{1}{1+e}, \frac{e}{1+e} \right] \quad (10)$$

$$\mathbf{h}_1 = \mathbf{a}_1 \mathbf{V}_1 = \begin{bmatrix} \frac{e}{1+e} & \frac{1}{1+e} \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{e}{1+e} & \frac{1-e}{1+e} \end{bmatrix} \quad (11)$$

$$\mathbf{h}_2 = \mathbf{a}_2 \mathbf{V}_2 = \begin{bmatrix} \frac{1}{1+e} & \frac{e}{1+e} \end{bmatrix} \begin{bmatrix} 0 & 0 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} \frac{-e}{1+e} & \frac{e}{1+e} \end{bmatrix} \quad (12)$$

$$\mathbf{z} = [\mathbf{h}_1, \mathbf{h}_2] = \frac{1}{1+e} [e, 1-e, -e, e] \quad (13)$$

2. Output dimension for time point 2 is 1×4 as shown in Equation (13)

For part 1:

- 3 pts for correct calculation of $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$.
- 2 pts for correct calculation of the softmax attention
- 2 pts for correct calculation of the embedding $\mathbf{z}_2$

22. (10 points) We have the following dataset of 4 (2D) points:

$$\mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & -2 \\ -1 & 1 \\ -2 & 1 \end{bmatrix}$$

We wish to cluster the points using K-Means with $K = 2$, i.e., two clusters. We initialize our clusters with $\mathbf{c}_1 = [1, -1]$ and $\mathbf{c}_2 = [-1, 1]$.

- (2 pts) Which points belongs to which cluster initially?
- (6 pts) After the first update of the centroids, what are the new values of the centroids?
- (2 pts) What do the centroids converge to and which points belong to which cluster?

Solution: The first two points belong to the first cluster. The last two points to the second cluster.

The updated value for the centroid of the first cluster is thus the average of the first two points thus $\mathbf{c}_1 = [1, -1.5]$ and the updated value for the centroid of the second cluster is thus the average of the last two points thus $\mathbf{c}_2 = [-1.5, 1]$.

After the update of the centroids, the first two points still belong to the first cluster and the second two points still belong to the second cluster, thus the algorithm has converged.

23. (8 points) You have an input sequence $\mathbf{x}_0 = [1, 0]$, $\mathbf{x}_1 = [0, 1]$. You are using a vanilla RNN such that $\mathbf{h}_{t+1} = \text{ReLU}(\mathbf{h}_t \mathbf{W}_h + \mathbf{x}_t \mathbf{W}_x + \mathbf{b})$. If

$$\mathbf{W}_x = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{W}_h = \begin{bmatrix} 0 & 2 \\ 0.5 & 0 \end{bmatrix}, \quad \mathbf{b} = [0, 0], \quad \mathbf{h}_0 = [0, 0]$$

- (6 pts) What is $\mathbf{h}_2$ equal to? Show your work.
- (2 pts) In seq-to-seq aligned prediction, the RNN outputs $y_t = \mathbf{h}_t \mathbf{w}_y$, with $\mathbf{w}_y = \begin{bmatrix} 2 \\ 0.5 \end{bmatrix}$, what is the output sequence (y_1, y_2) by this RNN on the input sequence (x_0, x_1) defined above?

Solution:

$$\mathbf{h}_1 = \text{ReLU}(0 + [1, 0] + 0) = [1, 0] \quad (14)$$

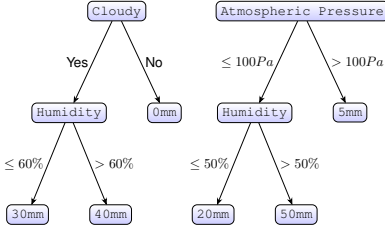
$$h_2 = \text{ReLU}([1, 0] \begin{bmatrix} 0 & 2 \\ 0.5 & 0 \end{bmatrix} + [0, 1]) = [0, 3] \quad (15)$$

$$y_1 = [1, 0] \begin{bmatrix} 2 \\ 0.5 \end{bmatrix} = 2 \quad (16)$$

$$y_2 = [0, 3] \begin{bmatrix} 2 \\ 0.5 \end{bmatrix} = 1.5 \quad (17)$$

So $(y_1, y_2) = (2, 1.5)$

24. (3 points) You trained a random forest of two trees to predict how much rain will fall in milliliters.



The first tree has a weight of 0.7 and the second tree has a weight of 0.3. If it is cloudy with a humidity level of 55% and an atmospheric pressure of 100 Pa, how many mm of rain do you predict?

Solution: $0.7 * 30\text{mm} + 0.3 * 50\text{mm} = 21\text{mm} + 15\text{mm} = 36\text{mm}$ of rain.

You may use this page as sketch.

You may use this page as sketch.